Snowflake
Training Notes

Innova Solutions — Internal Training Programme

12 Days · Core Concepts to Advanced Data Modelling

Prepared by: Satya | Data Engineer

Stack: Snowflake · SQL · dbt · Cloud Integrations

Day 1	Introduction to Snowflake
Day 2	SQL Operations — DDL, DML, DQL, Datatypes, Tables, Views
Day 3	Roles, RBAC, Stored Procedures, Streams, Tasks, Sequences, Shares
Day 4	Account Usage, Information Schema, Network Security & Policy
Day 5	File Formats & Stages (Internal & External)
Day 6	SQL Operations — Single-Row & Aggregate Functions
Day 7	SQL Operations — Subqueries, Set Operators, Window Functions
Day 8	SQL Operations — Joins
Day 9	Zero Copy Clone, Replication, Time Travel, Fail-Safe, Snowpipe
Day 10	Data Modelling — Normalization, OLTP/OLAP, Star/Snowflake Schema
Day 11	Storage Integration — Snowflake & AWS S3
Day 12	Data Modelling — Fact/Dimension Tables, SCD Types

Day 1 — Introduction to Snowflake

What is Snowflake?

Snowflake is a cloud-based data platform that provides **Data Warehouse as a Service (DWaaS / SaaS)**, designed for analytics and data management. It allows organisations to store, manage, and analyse large volumes of data efficiently.

Key Features

Cloud-Native Architecture

- Built specifically for the cloud — not ported from traditional databases.
- Runs on AWS, Azure, and Google Cloud Platform.
- Separates storage and compute so they scale independently.

Multi-Cluster Shared Data Architecture

- Supports concurrent workloads without performance bottlenecks.
- Multiple users/teams can query the same data simultaneously with isolated compute clusters.

Fully Managed (Maintenance-Free)

- No infrastructure management required (hardware, patches, etc.).
- Automatically handles tuning, backups, security, and availability.

Structured & Semi-Structured Data Support

- Handles CSV, JSON, Avro, Parquet, XML, and more.
- SQL-based querying with extensions for semi-structured data.

Secure Data Sharing

- Share live data across accounts without copying it (Secure Data Sharing).

Zero-Copy Cloning

- Instantly clone databases, schemas, or tables without duplicating storage.

Time Travel & Fail-Safe

- Restore data at any point within a configurable retention period (up to 90 days).

Snowflake Architecture

Database Storage Layer

- Stores table data and query results in **columnar format** optimised for querying.
- Data is organised into **micro-partitions** for enhanced performance.
- Snowflake autonomously manages organisation, file size, structure, compression, metadata, and statistics.
- Data objects are only accessible via SQL queries — not directly by customers.
- **Cluster keys** can be defined on large tables to improve query performance.

Query Processing Layer

- Handled by **Virtual Warehouses** — Snowflake's compute engines.
- Supports **scaling up** (increase warehouse size) and **scaling out** (add more clusters for concurrency).
- Compute costs are based on duration of query execution.
- **Auto-suspend** and **auto-resume** managed here — compute pauses when idle, resumes on demand.

Cloud Services Layer

- Coordinates and manages all operations across Snowflake.
- Handles authentication, access control, metadata management, and security.
- Manages serverless features: Snowpipe, materialized view maintenance, tasks, query parsing, optimisation, and access control.

Query Execution Flow

1. Query is submitted to a **Virtual Warehouse** (Query Processing Layer).
2. Passed to the **Cloud Services Layer** for authentication, access control, and parsing.
3. Services layer fetches required **metadata and data** from the Database Storage Layer.
4. An optimised **query execution plan** is generated.
5. The plan is sent back to the Virtual Warehouse for **execution**.
6. Results are returned to the user.

How to Access Snowflake

Signing Up (30-day Free Trial)

- Visit **https://signup.snowflake.com** and click *Start for Free*.
- Choose a cloud provider (AWS / Azure / GCP) and a geographic region.
- Fill out the signup form: name, email, company, username, and password.
- Verify your email via the activation link.
- Log in to **Snowsight** (Snowflake Web UI) and begin using SQL, loading data, and exploring features.

Snowsight — Quick Tour

Snowsight is Snowflake's modern web interface where you can perform data analysis, engineering tasks, monitor query activity, explore database objects, manage costs, and administer users and roles.

Worksheets

- Run ad hoc queries and DDL/DML operations.
- Write Snowpark Python code in Python worksheets.
- Review query history and results.
- Examine multiple worksheets, each with its own independent session.
- Export results for a selected statement while results are still available.

Snowflake Connectors & Drivers

Connector	Description
Web Interface (Snowsight)	Browser-based UI for managing all aspects of Snowflake.
SnowSQL	Command-line client providing full access to Snowflake functionality.
ODBC / JDBC Drivers	Standardised drivers for third-party tools like Tableau.
Native ETL Connectors	Built-in connectors for IBM DataStage, Informatica, and other ETL platforms.

Day 2 — SQL Operations — Part 1

SQL Categories

Category	Purpose	Key Commands
DDL — Data Definition Language	Define and manage database object structures.	CREATE, ALTER, DROP, TRUNCATE
DML — Data Manipulation Language	Manage data inside tables.	INSERT, UPDATE, DELETE, MERGE
DQL — Data Query Language	Retrieve data from the database.	SELECT, WHERE, GROUP BY, ORDER BY, HAVING
DCL — Data Control Language	Control access and permissions.	GRANT, REVOKE
TCL — Transaction Control Language	Manage transactions.	COMMIT, ROLLBACK, SAVEPOINT, SET TRANSACTION

Datatypes in Snowflake

Numeric

- **NUMBER(p,s)** / DECIMAL / NUMERIC — Fixed-point. *p* = total digits, *s* = decimal digits. Example: NUMBER(10,2) → 12345678.90

String / Text

- **TEXT** — Variable-length, no specific max.
- **VARCHAR(n)** — Variable-length up to *n* characters.
- **STRING** — Alias for VARCHAR.
- **CHAR** — Fixed-length, padded with spaces.

Date & Time

- **DATE** — YYYY-MM-DD
- **TIME** — HH:MI:SS
- **TIMESTAMP_NTZ** — No time zone.
- **TIMESTAMP_LTZ** — Local time zone.
- **TIMESTAMP_TZ** — Explicit time zone.

Constraints in Snowflake

Constraints enforce data integrity. Snowflake **enforces only the NOT NULL constraint** at the engine level; others (PK, FK, UNIQUE) are defined informally/for documentation.

Constraint	Description	Enforced by Snowflake?
PRIMARY KEY	Unique identifier; prevents NULLs.	No (informational only)
FOREIGN KEY	Links to a PK/UNIQUE column in another table.	No (informational only)
UNIQUE	All values distinct; allows NULLs.	No (informational only)
NOT NULL	Column must always have a value.	■ Yes — enforced

Databases & Schemas

- **Database** — Logical grouping of schemas. Each database belongs to a single Snowflake account.
- **Schema** — Logical grouping of database objects (tables, views, etc.). Each schema belongs to a single database.

Table Types in Snowflake

Type	Description	Time Travel / Fail-Safe	Use Case
Permanent	Standard persistent table.	Yes / Yes	General analytics, long-term storage.
Temporary	Session-scoped; auto-dropped on logout.	Yes / No	Intermediate session-level processing.
Transient	Persistent but no Fail-Safe.	Yes / No	Data needing moderate durability, low cost.
External	Points to data in S3/Azure/GCS.	No / No	Query external storage without loading.

Views in Snowflake

A **view** is a virtual table defined by a SQL query. It does not store data itself but simplifies complex queries and enforces security.

View Type	Description	Best For
Standard View	Query is executed in real-time each time the view is accessed.	Simple query abstraction.
Materialized View	Results are physically stored and periodically refreshed.	Performance on large, frequently queried datasets.

View Type	Description	Best For
Secure View	Hides the underlying SQL definition; restricts data access.	Sensitive/confidential data sharing.

Day 3 — Roles, Shares, Stored Procedures, Streams & Tasks

Roles in Snowflake (RBAC)

A **role** is a security object used to control access to database objects and manage user privileges via **Role-Based Access Control (RBAC)**.

Key Functions

- Roles are granted specific **privileges** (SELECT, INSERT, USAGE, etc.) on objects.
- Roles are **assigned to users**. The active role (current role) governs what a user can access.
- Roles can be **granted to other roles**, creating a privilege inheritance hierarchy.
- Recommended separation: **SYSADMIN** (object creation) | **SECURITYADMIN** (users/roles) | **PUBLIC** (minimal default access).

System-Defined Roles

Role	Responsibilities
ACCOUNTADMIN	Highest-level role; manages all account-level settings.
SYSADMIN	Creates and manages warehouses, databases, schemas, tables.
SECURITYADMIN	Manages users, roles, and privileges.
USERADMIN	Creates and manages users and roles (subset of SECURITYADMIN).
PUBLIC	Default role granted to all users; minimal access.

Stored Procedures

A **stored procedure** is a reusable block of SQL/procedural logic that performs a series of operations. Written in **Snowflake Scripting (SQL)** or **JavaScript**.

- Supports control-flow: IF, FOR, WHILE, TRY/CATCH.
- Can return values (e.g., status code, result set).
- Called repeatedly to perform data transformations, conditional logic, or DDL/DML.

Streams — Change Data Capture (CDC)

A **stream** is a CDC object that tracks changes (INSERT, UPDATE, DELETE) made to a table over time. A stream stores an *offset*, not data itself.

Stream Metadata Columns

Column	Description
METADATA$ACTION	DML operation recorded: INSERT or DELETE.
METADATA$ISUPDATE	TRUE if the change was part of an UPDATE (represented as DELETE + INSERT pair).
METADATA$ROW_ID	Unique, immutable row ID for tracking changes across offsets.

Types of Streams

Type	Tracks	Best For
Standard	INSERT, UPDATE, DELETE	Most CDC use cases.
Append-Only	INSERT only	Log data, sensor feeds, immutable fact tables.
Insert-Only	INSERT only (Iceberg / External tables)	External table ingestion.

Tasks — Scheduled SQL Execution

A **task** automates execution of SQL statements on a scheduled or event-driven basis. Tasks support **cron-based** or **interval-based** scheduling.

- Can run stored procedures for complex workflows.
- Supports **task chaining** — tasks can trigger other tasks (DAGs / Task Trees).
- Task Trees allow B-Tree-like structures: one root task triggers children sequentially.

Sequences

A **sequence** generates unique numeric values, typically used as surrogate keys or auto-incrementing primary keys.

- Customisable starting point, increment value, min/max values.
- Cycles back to start when the limit is reached (optional).
- Commonly used with INSERT statements to populate primary key fields.

Secure Data Sharing

A **share** allows one Snowflake account to share objects with another — *without copying or moving data*.

Shareable Objects

- Databases
- Tables
- Secure Views
- Secure Materialized Views

- Secure UDFs

Provider vs Consumer

Role	Responsibilities	Pays for...
Provider	Creates the share; owns the data.	Storage
Consumer	Creates a database from the share; reads data.	Compute (query costs)
Reader Account Consumer	Provider creates account for external consumers.	Everything (Provider pays all)

Day 4 — Account Usage, Information Schema & Network Security

Account Usage Schema

The **SNOWFLAKE.ACCOUNT_USAGE** schema enables querying object metadata and historical usage data for your account (and reader accounts if any).

Key Characteristics

- Available only to users with access to the shared SNOWFLAKE database.
- Covers up to **1 year** of history for some views.
- Ideal for account-wide auditing, billing monitoring, and security analysis.

Common Queries

```
SELECT * FROM SNOWFLAKE.ACCOUNT_USAGE.LOGIN_HISTORY;
SELECT * FROM SNOWFLAKE.ACCOUNT_USAGE.QUERY_HISTORY;
```

Information Schema

The **INFORMATION_SCHEMA** is a set of metadata views at the database and schema level. Each database has its own INFORMATION_SCHEMA.

- Contains views for all objects in the database, plus account-level objects (roles, warehouses, databases).
- Includes table functions for historical and usage data.

Example

```
SELECT * FROM INFORMATION_SCHEMA.COLUMNS
WHERE TABLE_SCHEMA = 'MY_SCHEMA' AND TABLE_NAME = 'MY_TABLE';
```

Account Usage vs Information Schema

Aspect	ACCOUNT_USAGE	INFORMATION_SCHEMA
Scope	Account-wide	Per-database
Latency	Up to 120 mins	Near real-time
History	Up to 1 year	Limited (7 days for some)
Best For	Auditing, billing, security	Object metadata, query optimisation

Network Security & Policy

Network Policies

- **IP Whitelisting / Blacklisting** — Allow or block specific IP address ranges.
- **Account-Level Policy** — Applies to all users and services unless overridden.
- **User-Level Policy** — Can override the account policy for specific users.

Authentication Methods

Method	Description
Username / Password	Default authentication.
Multi-Factor Authentication (MFA)	Additional OTP/push notification layer.
Federated Authentication & SSO	Integrate with IdP (Okta, Azure AD, etc.).
OAuth	For API-based access with token-based auth.
Key Pair Authentication	Recommended for service accounts and automated pipelines.

Day 5 — File Formats & Stages

File Formats

A **Snowflake File Format** is a named database object that encapsulates file type and formatting options, simplifying bulk load/unload operations.

Supported Formats

- **CSV** — Delimited flat files (most common for tabular data).
- **JSON** — Semi-structured, document-style data.
- **AVRO** — Binary format with schema embedded in the file.
- **ORC** — Optimised columnar format for Hadoop ecosystems.
- **PARQUET** — Columnar format widely used in big-data stacks.
- **XML** — Hierarchical, tag-based semi-structured data.

Stages in Snowflake

A **stage** is a location where data files are stored ('staged') for loading into or unloading from Snowflake tables.

Internal Stages

Managed by Snowflake itself. No external setup required. Used to load data from a local drive.

Stage Type	Scope	Notes
User Stage	Per-user	Only accessible by the owning user. Referenced as @~.
Table Stage	Per-table	Tied to a specific table. Referenced as **@%table_name**.
Named Stage	Shared	Independent object; reusable across tables and users.

External Stages

Reference files in cloud storage outside Snowflake (Amazon S3, Azure Blob Storage, or Google Cloud Storage) via a **Storage Integration**.

- **Storage Integration** — Snowflake object that stores IAM credentials for the external cloud provider.

Data Loading & Unloading Commands

Command	Direction	Description
PUT	Local → Internal Stage	Upload files from your local filesystem to an internal stage.
GET	Internal Stage → Local	Download files from an internal stage to your local filesystem.

Command	Direction	Description
COPY INTO	Stage → Table (or reverse)	Bulk load data from a stage into a table, or unload a table into a stage.

Day 6 — SQL Operations — Part 2

Single-Row Functions

Single-row functions operate on one row at a time and return one result per row. Used in SELECT, WHERE, and other expressions.

Function	Description
UPPER / LOWER	Converts string to upper / lowercase.
SUBSTR	Extracts a substring by position and length.
INSTR	Returns position of a substring within a string.
LPAD / RPAD	Pads a string on left / right to a specified length.
LTRIM / RTRIM	Removes leading / trailing characters (default: spaces).
REPLACE	Replaces all occurrences of a substring with another.
TRANSLATE	Character-to-character mapping replacement.
NVL	Replaces NULL with a specified value.
NULLIF	Returns NULL if two expressions are equal; else returns first.
COALESCE	Returns the first non-NULL value from a list.
ROUND	Rounds numeric value to specified decimal places.
TRUNC	Truncates numeric or date value to specified precision.
FLOOR / CEIL	Largest integer ≤ value / Smallest integer ≥ value.
MONTHS_BETWEEN	Number of months between two dates.
TO_DATE	Converts string to DATE using a format.
NEXT_DAY	Date of the next specified weekday after a given date.
TO_CHAR	Converts date or number to string using a format.

CASE Statement

Implements conditional logic in SQL — equivalent to IF-THEN-ELSE. Returns specific values based on evaluated conditions.

```
CASE
  WHEN condition1 THEN result1
```

```
  WHEN condition2 THEN result2
  ELSE default_result
END
```

Aggregate Functions

Perform a calculation on a *set* of values and return a single summary value. Commonly used with GROUP BY.

Function	Description
COUNT()	Counts rows. COUNT(*) = all rows; COUNT(col) = non-NULL values only.
SUM()	Returns total sum of a numeric column.
AVG()	Returns the arithmetic mean of numeric values.
MIN()	Returns the minimum value in a set.
MAX()	Returns the maximum value in a set.

Aggregation Query Clause Order

Order	Clause	Purpose
1	FROM	Identify the source table/object.
2	WHERE	Filter rows before grouping.
3	GROUP BY	Group rows by column(s).
4	HAVING	Filter groups after aggregation.
5	ORDER BY	Sort final result set (ASC or DESC).

Day 7 — SQL Operations — Part 3

Subqueries

A **subquery** is a SQL query nested inside another query. It returns data used as a condition to restrict the data retrieved by the outer query.

Type	Returns	Description
Scalar Subquery	Single value	Returns exactly one column, one row. Used in SELECT or WHERE.
Single-Row Subquery	One row (1+ columns)	Returns one row. Used with =, <, > operators.
Multi-Row Subquery	Multiple rows	Returns many rows. Used with IN, ANY, ALL.
Correlated Subquery	Varies	References columns from the outer query; executes once per outer row.

Set Operators

Set operators combine results of two or more SELECT statements. Both queries must have the same number of columns with compatible data types.

Operator	Description
UNION	Combines results from both queries, **removing duplicates**.
UNION ALL	Combines results **including duplicates** (faster than UNION).
INTERSECT	Returns **only rows common** to both queries.
EXCEPT	Returns rows from the **first query not present** in the second.

Analytical (Window) Functions

An **analytical function** calculates across a *window* of rows related to the current row, returning a value *per row* (unlike aggregate functions which collapse rows).

OVER() Clause Components

- **PARTITION BY** — Divides result set into groups (like GROUP BY but keeps all rows).
- **ORDER BY** — Defines row order within each partition.
- **ROWS BETWEEN** — (Optional) Narrows the window to a specific range of rows.

Common Window Functions

Function	Description
ROW_NUMBER()	Assigns a unique sequential number to each row within a partition.
RANK()	Assigns rank; ties get the same rank, next rank skips (gaps).
DENSE_RANK()	Like RANK() but no gaps after ties.
LAG(col, n)	Accesses the value of a column *n* rows **before** the current row.
LEAD(col, n)	Accesses the value of a column *n* rows **after** the current row.

Day 8 — SQL Operations — Part 4: Joins

Joins in SQL

Joins combine data from two or more tables based on a related column, enabling retrieval of data that spans multiple tables.

Join Type	Returns	NULL Behaviour
INNER JOIN	Only rows with matching values in both tables.	No NULLs from either side.
LEFT JOIN	All rows from the left table + matching rows from right.	NULLs for unmatched right-side columns.
RIGHT JOIN	All rows from the right table + matching rows from left.	NULLs for unmatched left-side columns.
FULL JOIN	All rows from both tables.	NULLs where no match exists on either side.
CROSS JOIN	Cartesian product — every row from left × every row from right.	Not applicable.
SELF JOIN	A table joined with itself (useful for hierarchical data).	Depends on join type used.

Day 9 — Zero Copy Clone, Time Travel, Fail-Safe & Snowpipe

Zero-Copy Clone

Zero-Copy Clone creates a copy of a table, schema, or database **without duplicating storage**. The clone shares underlying storage with the original. Changes to either object create new storage independently.

Cloneable Objects

Category	Objects
Data Containment	Databases, Schemas, Tables, Streams
Configuration & Transformation	Stages, File Formats, Sequences, Tasks

■ *Internal named stages cannot be cloned.*

Replication

Replication copies and synchronises data across different Snowflake accounts or regions (including multi-cloud: AWS, Azure, GCP).

- **Primary Database** — Enabled for replication; source of truth. Supports all DML/DDL.
- **Secondary Database** — Read-only replica. Must be refreshed periodically from the primary.
- Replication is supported across regions and cloud platforms within the same organisation.

Time Travel

Time Travel allows querying, cloning, restoring, or recovering data at any point within a retention period.

Retention Period

- Standard Edition: up to 1 day.
- Enterprise Edition and above: up to 90 days (configurable).

Use Cases

- Restoring accidentally deleted/modified data.
- Examining data changes over time.
- Cloning data at a previous state.
- Recovering dropped tables or schemas.

Query Options

```
-- By timestamp
SELECT * FROM my_table AT (TIMESTAMP => '2025-01-01 09:00:00'::TIMESTAMP);
-- By offset (seconds ago)
SELECT * FROM my_table AT (OFFSET => -3600);
-- By statement ID
SELECT * FROM my_table BEFORE (STATEMENT => '<query_id>');
-- Restore dropped table
UNDROP TABLE my_table;
```

Fail-Safe

Fail-Safe is a **last-resort** data recovery mechanism managed by Snowflake Support — not directly accessible by users.

Aspect	Details
Retention Period	7 days after the Time Travel period ends.
Access	Cannot be queried directly. Requires Snowflake Support intervention.
Use Case	Recovery after Time Travel period expires or objects are permanently dropped.
Cost	Billed separately based on data stored in Fail-Safe.
Availability	All editions; Enterprise/Business Critical may have enhanced SLAs.

Time Travel vs Fail-Safe

Feature	Time Travel	Fail-Safe
Max Duration	90 days (Enterprise+)	7 days (after Time Travel)
User Access	■ Full SQL access	■ Snowflake Support only
Cost	Included (edition-based)	Additional storage billing
Recovery Speed	Instant via SQL	Slow — manual process with support
Purpose	Self-service recovery	Last-resort disaster recovery

Snowpipe — Continuous Data Ingestion

Snowpipe automatically loads data from files as soon as they arrive in a stage. Ideal for **near-real-time ingestion** without batch waits.

Key Features

- **Automated** — Detects new files and loads them using a predefined COPY INTO command.

- **Serverless & Scalable** — Snowflake manages infrastructure and scales automatically.
- **Event-Based** — Uses cloud storage event notifications (AWS S3 Events, Azure Event Grid).
- **Micro-Batching** — Loads data in small batches for near-real-time availability.

How Snowpipe Works

1. Data file lands in a stage (S3, Azure Blob, GCS).
2. Cloud event notification is sent to Snowpipe.
3. Snowpipe executes the predefined COPY INTO statement.
4. Data becomes available in the Snowflake table almost immediately.

Day 10 — Data Modelling — Normalization, OLTP vs OLAP

Normalization

Normalization organises data into tables and columns to **eliminate redundancy**, ensure data dependencies make sense, and improve data integrity and consistency.

Normal Form	Rule	Eliminates
1NF	Each column has atomic (indivisible) values; a primary key exists.	Repeating groups, multi-valued columns.
2NF	Meets 1NF + no partial dependencies (every non-key column depends on the full PK).	Partial dependency on composite PKs.
3NF	Meets 2NF + no transitive dependencies (non-key column depending on another non-key column).	Transitive dependencies.
BCNF	Stricter 3NF — every determinant must be a candidate key.	Anomalies not caught by 3NF.

Denormalization

Adding redundant data to improve **read performance** for complex queries. Done strategically after normalization, typically in analytical (OLAP) systems.

Normalization vs Denormalization

Aspect	Normalization	Denormalization
Purpose	Reduce redundancy	Improve read performance
Storage	Efficient	Increased (duplicate data)
Query Speed	Slower (more joins)	Faster (fewer joins)
Maintenance	Easier	Harder (data redundancy)
System Type	OLTP	OLAP / Data Warehousing

OLTP vs OLAP

Feature	OLTP	OLAP
Purpose	Transaction processing	Data analysis & reporting
Data Volume	Small, real-time	Large, historical
Operations	Read & write (INSERT/UPDATE)	Mostly read (complex queries)

Feature	OLTP	OLAP
Speed	High — simple queries	High — complex aggregations
Schema	Normalised	Denormalised (Star/Snowflake)
Users	Clerks, admins, customers	Analysts, executives
Examples	ATM, POS, bookings	BI dashboards, sales trends

Database vs Data Warehouse in Snowflake

Feature	Database (in Snowflake)	Virtual Warehouse (Compute)
Purpose	Organise and store data	Provide compute power to process queries
Contains	Schemas, tables, views, objects	Compute resources only (no data stored)
Scalable?	Not applicable	Yes — scale up/down as needed
Billing	Storage costs	Compute usage (per-second billing)
Example Use	Store sales, marketing data	Run ETL jobs, dashboards, ad hoc queries

Day 11 — Storage Integration — Snowflake & AWS S3

Step-by-Step: Snowflake ↔ AWS S3 Integration

1. AWS Setup

- Create an **AWS account** (or use existing).
- Go to **S3** → Create a bucket → Create a folder → Upload your data file.
- Go to **IAM** → Create a Role → Attach **S3 Full Access** policy.
- Navigate to the new role → Copy the **ARN** number.

2. Create Storage Integration in Snowflake

```
CREATE STORAGE INTEGRATION my_s3_int
  TYPE = EXTERNAL_STAGE
  STORAGE_PROVIDER = 'S3'
  ENABLED = TRUE
  STORAGE_ALLOWED_LOCATIONS = ('s3://your-bucket/your-folder/');
```

- Run **DESC INTEGRATION my_s3_int** and copy **STORAGE_AWS_IAM_USER_ARN**.

3. Update AWS Trust Policy

- Go to **IAM Role** → **Trust Relationships** → **Edit Trust Policy**.
- Replace the AWS principal with the **STORAGE_AWS_IAM_USER_ARN** value copied from Snowflake.

4. Create External Stage & Load Data in Snowflake

```
CREATE STAGE my_s3_stage
  STORAGE_INTEGRATION = my_s3_int
  URL = 's3://your-bucket/your-folder/'
  FILE_FORMAT = (TYPE = 'CSV');
```

```
LIST @my_s3_stage; -- Verify files in the stage
```

```
COPY INTO my_table FROM @my_s3_stage; -- Load data into table
```

Day 12 — Data Modelling — Fact/Dimension Tables, Star/Snowflake Schema, SCD

Fact Tables

A **fact table** is the central table in a star/snowflake schema. It stores **quantitative, measurable data** (facts) and foreign keys to dimension tables.

Key Characteristics

- Stores **measures**: numeric values that can be aggregated (revenue, quantity, profit).
- Contains **foreign keys** referencing dimension tables.
- High granularity — typically one row per event/transaction.
- Examples: *sales_fact, orders_fact, inventory_fact*

Dimension Tables

A **dimension table** contains **descriptive, contextual attributes** about the facts (e.g., who, what, when, where).

Key Characteristics

- Stores descriptive attributes: names, categories, dates, locations.
- Has a **primary key** uniquely identifying each entity.
- Used for **filtering and grouping** measures in queries.
- Examples: *customer_dim, product_dim, date_dim, store_dim*
- **Slowly Changing Dimensions (SCD)** — dimension data that changes slowly over time (e.g., customer address change).

Fact vs Dimension Tables

Feature	Fact Table	Dimension Table
Purpose	Quantitative / measurable data	Descriptive / contextual data
Data Type	Mostly numeric (measures)	Mostly text / categorical
Granularity	High — one row per event	Low — one row per entity
Keys	Foreign keys to dimensions	Primary key
Query Use	Aggregation: SUM(), COUNT()	Filtering, grouping, labelling
Update Freq.	Frequent (e.g., daily)	Infrequent (slowly changing)

Feature	Fact Table	Dimension Table
Examples	sales_fact, orders_fact	customer_dim, product_dim

Star Schema vs Snowflake Schema

Feature	Star Schema	Snowflake Schema
Structure	Fact table surrounded by flat dimensions	Fact + normalised, multi-level dimensions
Joins Required	Fewer — simpler queries	More — dimensions split into sub-tables
Data Redundancy	Higher (denormalised dims)	Lower (normalised dims)
Query Performance	Generally faster	Slightly slower due to extra joins
Storage	More	Less
Best For	Simpler BI dashboards	Complex, large-scale data warehouses

Slowly Changing Dimensions (SCD)

SCD describes how to handle dimension attributes that change gradually over time (e.g., customer address, product category) in a data warehouse.

Type	Strategy	Use Case
Type 0	No changes tracked; data remains static.	Values that should never change.
Type 1	Overwrite old data with new.	History not important; only current value matters.
Type 2	Add a new row with new values; preserve old row.	Full history tracking required (most common).
Type 3	Add a column for previous value alongside current.	Only current and one previous value needed.
Type 4	Maintain history in a separate history table.	History tracking isolated from main dimension.
Type 6	Hybrid of Types 1, 2, and 3.	Complex scenarios needing multiple strategies.

Lab Exercises (Practice)

1. Undrop a table using Time Travel.
2. Reload a truncated table using Time Travel.
3. Create a sequence with increment of 1 and use it as a primary key in a table.
4. Create 2 stored procedures to insert last month's data from one table to another:
    - Proc A: New record → INSERT; Existing record → DELETE then INSERT.
    - Proc B: New record → INSERT; Existing record → UPDATE.
5. Create 2 tasks to run the above procedures on the **1st of every month**.

6. Create a Zero-Copy Clone of a table.

■ Interview Prep Additions — Critical Topics Not in Training Notes

The following topics are **frequently asked in Snowflake Data Engineer interviews**. They were not part of the 12-day training but are essential for job preparation.

A. Virtual Warehouse — Sizing, Scaling & Best Practices

Your training notes mentioned Virtual Warehouses as compute engines but never covered sizing, multi-cluster settings, or cost optimisation — all commonly asked in interviews.

Warehouse T-Shirt Sizes

Size	Credits / Hour	Best For
X-Small (XS)	1	Development, light ad hoc queries.
Small (S)	2	Small workloads, single-user queries.
Medium (M)	4	Moderate ETL, small team workloads.
Large (L)	8	Heavy ETL, medium concurrency.
X-Large (XL)	16	Complex transformations, large datasets.
2X-Large	32	Very large batch jobs.
3X-Large	64	Massive parallel workloads.
4X-Large	128	Extreme scale — rarely needed.

■ *Billing is per-second with a 60-second minimum. Auto-suspend saves cost when idle.*

Scaling Up vs Scaling Out

Type	How	When to Use
Scale Up	Increase warehouse size (S → L)	Query is slow — needs more compute per query.
Scale Out	Add more clusters (Multi-cluster)	Many concurrent users — queries are queuing.

Key Warehouse SQL

```
-- Create a warehouse with auto-suspend after 5 mins idle
CREATE WAREHOUSE my_wh
  WAREHOUSE_SIZE = 'MEDIUM'
  AUTO_SUSPEND = 300
  AUTO_RESUME = TRUE
  MIN_CLUSTER_COUNT = 1
  MAX_CLUSTER_COUNT = 3; -- Multi-cluster (Enterprise+)
```

```
ALTER WAREHOUSE my_wh SUSPEND; -- Manually suspend
ALTER WAREHOUSE my_wh RESUME; -- Manually resume
ALTER WAREHOUSE my_wh SET WAREHOUSE_SIZE = 'LARGE'; -- Resize
```

B. Query Result Cache

One of Snowflake's most frequently asked interview topics. Snowflake automatically caches query results and reuses them — completely free, no warehouse needed.

- Results are cached for **24 hours** in the Cloud Services Layer.
- If the **same query** is re-run and the underlying data has **not changed**, Snowflake returns the cached result instantly.
- **No virtual warehouse credits** are consumed for a cache hit.
- Cache is **per-user and per-role** — other users don't automatically share your cache.
- Cache is invalidated when the underlying table data changes (DML operations).
- You can disable result caching with: **ALTER SESSION SET USE_CACHED_RESULT = FALSE;**

3 Layers of Caching in Snowflake

Cache Layer	Where	Duration	Description
Result Cache	Cloud Services Layer	24 hours	Exact same query + unchanged data → free instant result.
Metadata Cache	Cloud Services Layer	Persistent	Table stats, schema info — used for query optimisation.
Local Disk Cache	Virtual Warehouse	Session	Raw data micro-partitions cached on SSD of compute node.

C. Micro-Partitions & Clustering Keys

Your notes mention micro-partitions in Day 1 but never explain how they work. This is a very common deep-dive interview question.

What are Micro-Partitions?

- Snowflake stores table data in **micro-partitions** — contiguous units of storage between **50 MB and 500 MB** of uncompressed data.
- Each micro-partition stores data in **columnar format** and is automatically compressed.
- Snowflake maintains **metadata** for each micro-partition: min/max values per column, NULL count, distinct value count.
- This metadata enables **partition pruning** — Snowflake skips micro-partitions that don't match the query filter.
- Pruning = fewer micro-partitions scanned = faster queries = lower cost.

Clustering Keys

- By default, data is stored in **insertion order** across micro-partitions.
- For very large tables, you can define a **clustering key** to physically co-locate similar data.
- Snowflake uses **Automatic Clustering** to maintain the clustering over time as data is inserted.
- Best applied on columns used frequently in **WHERE, JOIN, or ORDER BY** clauses.
- Not recommended for small tables — overhead outweighs benefit.

```
-- Define clustering key on a large table
ALTER TABLE sales_fact CLUSTER BY (sale_date, region);
```

■ *Interview tip: Explain the difference between clustering keys and indexes. Snowflake has NO traditional indexes — micro-partition pruning + clustering keys replace them.*

D. Snowflake Editions

Snowflake editions determine which features are available. Time Travel limits, multi-cluster warehouses, and security features all depend on edition.

Edition	Time Travel	Multi-Cluster WH	Key Additional Features
Standard	1 day	■ No	Core features, basic security.
Enterprise	90 days	■ Yes	Multi-cluster WH, column-level security, materialized views.
Business Critical	90 days	■ Yes	HIPAA, PCI-DSS, enhanced encryption, private link.
Virtual Private (VPS)	90 days	■ Yes	Dedicated Snowflake environment, highest isolation.

E. GRANT & REVOKE — Privilege Management

Your notes covered DCL (GRANT/REVOKE) in Day 2 and RBAC in Day 3 but never showed actual syntax. This is always asked in interviews.

Common GRANT Syntax

```
-- Grant privilege on an object to a role
GRANT SELECT ON TABLE my_schema.my_table TO ROLE analyst_role;

-- Grant usage on schema and database (required before table-level grants)
GRANT USAGE ON DATABASE my_db TO ROLE analyst_role;
GRANT USAGE ON SCHEMA my_db.my_schema TO ROLE analyst_role;

-- Grant role to a user
GRANT ROLE analyst_role TO USER satya;

-- Grant role to another role (role hierarchy)
GRANT ROLE analyst_role TO ROLE sysadmin;
```

Common REVOKE Syntax

```
-- Revoke a privilege
REVOKE SELECT ON TABLE my_schema.my_table FROM ROLE analyst_role;

-- Revoke role from user
REVOKE ROLE analyst_role FROM USER satya;
```

Common Privileges

Privilege	Applies To	Description
USAGE	Database, Schema, Warehouse	Allows access to the object (required before lower-level access).
SELECT	Table, View	Read data.
INSERT	Table	Add rows.
UPDATE	Table	Modify rows.
DELETE	Table	Remove rows.
CREATE TABLE	Schema	Create tables inside a schema.
OWNERSHIP	Any object	Full control — can grant to others.
OPERATE	Warehouse	Suspend/resume a warehouse.
MONITOR	Warehouse	View query history and usage.

F. MERGE Statement (Upsert)

MERGE was listed under DML in Day 2 but never explained. It is heavily used in ETL pipelines and is a very common interview + practical question.

MERGE combines INSERT, UPDATE, and DELETE into a single statement. Also called an **UPSERT** (Update + Insert).

Syntax

```
MERGE INTO target_table AS t
USING source_table AS s
ON t.id = s.id
WHEN MATCHED AND s.status = 'DELETED' THEN
  DELETE
WHEN MATCHED THEN
  UPDATE SET t.name = s.name, t.salary = s.salary
WHEN NOT MATCHED THEN
  INSERT (id, name, salary) VALUES (s.id, s.name, s.salary);
```

MERGE with Streams (Common Pattern)

The most common real-world use: process a Stream's CDC output into a target table.

```
MERGE INTO customer_dim AS t
USING customer_stream AS s
ON t.customer_id = s.customer_id
WHEN MATCHED AND s.METADATA$ACTION = 'DELETE' THEN DELETE
WHEN MATCHED AND s.METADATA$ACTION = 'INSERT' THEN
  UPDATE SET t.email = s.email
WHEN NOT MATCHED AND s.METADATA$ACTION = 'INSERT' THEN
  INSERT (customer_id, email) VALUES (s.customer_id, s.email);
```

■ *Interview tip: Streams + Tasks + Merge is the classic Snowflake ELT pipeline pattern. Be ready to explain this end-to-end.*

G. Semi-Structured Data — VARIANT, PARSE_JSON & FLATTEN

Your notes say Snowflake supports JSON, Avro, Parquet etc. but never show how to actually query semi-structured data. This is very important for a Data Engineer role.

VARIANT Data Type

- VARIANT is Snowflake's universal data type for storing semi-structured data (JSON, XML, Avro, Parquet).
- A VARIANT column can store any JSON structure — objects, arrays, strings, numbers.
- Snowflake automatically parses and stores VARIANT data in an optimised columnar format internally.

Querying JSON with Dot Notation & Colon Syntax

```
-- Sample data in VARIANT column 'data':
-- { 'customer': { 'name': 'Satya', 'city': 'Hyderabad' }, 'orders': [1,2,3] }
```

```
-- Colon syntax to access a field
SELECT data:customer:name::STRING AS customer_name FROM my_table;

-- Dot notation (alternative)
SELECT data['customer']['city']::STRING AS city FROM my_table;

-- Access array element by index
SELECT data:orders[0]::INT AS first_order FROM my_table;
```

PARSE_JSON — Convert String to VARIANT

```
SELECT PARSE_JSON('{"name": "Satya", "age": 25}'):name::STRING;
```

FLATTEN — Explode Arrays into Rows

FLATTEN is used to convert nested arrays inside a VARIANT column into individual rows.

```
SELECT f.value::STRING AS order_id
FROM my_table,
LATERAL FLATTEN(INPUT => data:orders) f;
```

■ *Interview tip: If asked 'how do you work with JSON in Snowflake?' — mention VARIANT type, colon/dot notation for extraction, PARSE_JSON for strings, and FLATTEN for arrays.*

H. Dynamic Data Masking & Row Access Policies

These are Enterprise-level security features important for healthcare, finance, and compliance-heavy industries — very relevant for your Centene/Medicaid project experience.

Dynamic Data Masking

Masks sensitive column values (PII, PHI) dynamically at query time based on the user's role. The data is stored unmasked — masking happens only when displayed.

```
-- Step 1: Create a masking policy
CREATE MASKING POLICY mask_ssn AS
  (val STRING) RETURNS STRING ->
  CASE
    WHEN CURRENT_ROLE() IN ('ANALYST') THEN '***-**-' || RIGHT(val, 4)
    WHEN CURRENT_ROLE() IN ('ADMIN') THEN val -- full value
    ELSE '***MASKED***'
  END;

-- Step 2: Apply the policy to a column
ALTER TABLE patient_data MODIFY COLUMN ssn
  SET MASKING POLICY mask_ssn;
```

Row Access Policies

Controls which **rows** a user or role can see — row-level security.

```
-- Create a row access policy
CREATE ROW ACCESS POLICY region_policy AS
  (region STRING) RETURNS BOOLEAN ->
  CURRENT_ROLE() = 'ADMIN'
  OR region = CURRENT_USER();

-- Apply to a table
ALTER TABLE sales_data ADD ROW ACCESS POLICY region_policy ON (region);
```

■ *Key difference: Dynamic Data Masking hides column values. Row Access Policies hide entire rows. Both work transparently at query time.*

I. Snowflake-Specific SQL Functions

These are functions unique to or particularly important in Snowflake — not covered in standard SQL training.

IFF() — Inline IF

Snowflake shorthand for a simple two-outcome CASE statement.

```
-- IFF(condition, value_if_true, value_if_false)
SELECT IFF(salary > 50000, 'High', 'Low') AS salary_band FROM employees;
```

ZEROIFNULL() & NULLIFZERO()

```
SELECT ZEROIFNULL(NULL); -- Returns 0
SELECT NULLIFZERO(0); -- Returns NULL
```

TRY_CAST & TRY_TO_DATE — Safe Type Conversion

Safe conversion — returns NULL instead of throwing an error if the conversion fails.

```
SELECT TRY_CAST('abc' AS INT); -- Returns NULL (no error)
SELECT TRY_TO_DATE('2025-13-01', 'YYYY-MM-DD'); -- Returns NULL (invalid month)
```

QUALIFY — Filter Window Function Results

QUALIFY filters rows based on the result of a window function — like HAVING but for analytical functions.

```
-- Get only the latest record per customer (without subquery)
SELECT customer_id, order_date, amount
FROM orders
QUALIFY ROW_NUMBER() OVER (PARTITION BY customer_id ORDER BY order_date DESC) = 1;
```

■ *Without QUALIFY, you'd need a subquery or CTE. QUALIFY makes it cleaner and is Snowflake-specific.*

GENERATOR — Generate Rows

```
-- Generate 10 rows (useful for testing)
SELECT SEQ4() AS id FROM TABLE(GENERATOR(ROWCOUNT => 10));
```

Additional String Functions (Missing from Day 6)

Function	Description	Example
TRIM(str)	Remove leading and trailing spaces.	TRIM(' hello ') → 'hello'
LENGTH(str)	Return number of characters in string.	LENGTH('Satya') → 5
CONCAT(a, b)	Concatenate two strings.	CONCAT('Snow', 'flake') → 'Snowflake'
SPLIT(str, delim)	Split string into array by delimiter.	SPLIT('a,b,c', ',') → ['a','b','c']
POSITION(sub, str)	Find position of substring.	POSITION('flake' IN 'Snowflake') → 5
LEFT(str, n)	Extract n chars from left.	LEFT('Snowflake', 4) → 'Snow'
RIGHT(str, n)	Extract n chars from right.	RIGHT('Snowflake', 5) → 'flake'
REGEXP_LIKE	Test if string matches a regex pattern.	REGEXP_LIKE(email, '.*@.*\.com')

J. Query Profile & Performance Tuning

Query Profile is Snowflake's built-in query execution visualiser in Snowsight. It helps diagnose slow queries — a practical skill interviewers often test.

How to Access Query Profile

- Run a query in Snowsight → Click the query in History → Click **Query Profile** tab.
- Shows a visual node-graph of the query execution plan with time spent per step.

Key Things to Look For

Indicator	What It Means	Fix
Spillage to disk	Data too large for memory — warehouse too small.	Scale up warehouse size.
TableScan (large %)	Full table scan — no pruning.	Add WHERE clause on clustered/filtered column.
Partitions scanned = total partitions	No partition pruning happening.	Check clustering key or rewrite filter.
Join explosion	Cartesian product or bad join condition.	Verify join keys; avoid CROSS JOIN accidentally.
Many small files	Inefficient micro-partition usage.	Use COPY INTO with larger batch files.

Performance Tuning Checklist

- **Right-size your warehouse** — start small, scale up only if needed.
- **Use Result Cache** — avoid re-running identical queries unnecessarily.
- **Partition pruning** — always filter on clustering key columns in WHERE clause.
- **Avoid SELECT *** — Snowflake is columnar; only fetch columns you need.
- **Use transient tables** for staging/intermediate data to avoid Fail-Safe storage costs.
- **Suspend idle warehouses** — use AUTO_SUSPEND = 60 (1 minute) for dev warehouses.
- **Use COPY INTO over INSERT** for bulk data loading — much faster.
- **Cluster large tables** on frequently filtered columns.

K. User-Defined Functions (UDFs)

UDFs are reusable custom functions — closely related to Stored Procedures but with key differences. A common interview question is: *What is the difference between a UDF and a Stored Procedure?*

UDF vs Stored Procedure

Feature	UDF	Stored Procedure
Returns	A single value or table	A single value (optional)

Feature	UDF	Stored Procedure
Used In	SELECT, WHERE, expressions	Called with CALL — not in SELECT
DML Operations	■ Cannot perform DML	■ Can INSERT, UPDATE, DELETE
Purpose	Custom calculations / transforms	Workflow automation, ETL logic
Languages	SQL, JavaScript, Python, Java	SQL Scripting, JavaScript, Python, Java

SQL UDF Example

```
-- Create a simple UDF to calculate tax
CREATE FUNCTION calculate_tax(amount FLOAT, rate FLOAT)
RETURNS FLOAT
LANGUAGE SQL
AS
'amount * rate / 100';

-- Use in a query
SELECT product, price, calculate_tax(price, 18) AS gst FROM products;
```

JavaScript UDF Example

```
CREATE FUNCTION mask_email(email STRING)
RETURNS STRING
LANGUAGE JAVASCRIPT
AS
$$
  var parts = EMAIL.split('@');
  return parts[0].substring(0,2) + '***@' + parts[1];
$$;
```

L. Common Table Expressions (CTEs)

CTEs (WITH clause) were not in your training notes but are used constantly in real SQL work and interviews.

A CTE is a temporary named result set defined at the beginning of a query. It makes complex queries more readable and reusable within the same query.

Basic CTE Syntax

```
WITH monthly_sales AS (
  SELECT region, SUM(amount) AS total
  FROM sales
  WHERE MONTH(sale_date) = MONTH(CURRENT_DATE())
  GROUP BY region
)
SELECT region, total
```

```
FROM monthly_sales
WHERE total > 100000
ORDER BY total DESC;
```

Multiple CTEs

```
WITH
raw_data AS (
  SELECT * FROM orders WHERE status = 'COMPLETE'
),
aggregated AS (
  SELECT customer_id, SUM(amount) AS total FROM raw_data GROUP BY 1
)
SELECT * FROM aggregated WHERE total > 5000;
```

CTE vs Subquery — When to Use Which

Aspect	CTE (WITH clause)	Subquery
Readability	High — named, defined once at top	Low — nested, harder to follow
Reusability	Can reference same CTE multiple times	Must repeat the subquery each time
Debugging	Easy — run CTE block independently	Harder to isolate
Performance	Similar (Snowflake optimises both)	Similar
Best For	Complex multi-step logic	Simple one-off filters

M. Classic Snowflake ELT Pipeline Pattern

Interviewers frequently ask: *How would you design an end-to-end data pipeline in Snowflake?* This section ties together Streams, Tasks, MERGE, and Snowpipe into a complete answer.

Pattern: Snowpipe → Stream → Task → MERGE

Step	Component	What Happens
1	External Stage (S3)	Raw files land in S3 bucket from source system.
2	Snowpipe	Detects new file → auto-loads into RAW / staging table.
3	Stream	Stream on staging table captures INSERT/UPDATE/DELETE changes (CDC).
4	Task (scheduled)	Runs every N minutes — triggers MERGE statement.
5	MERGE Statement	Applies stream changes to the final target/dimension table (upsert logic).
6	Target Table	Clean, transformed, up-to-date table ready for analytics / Power BI.

Full Example SQL

```
-- Step 1: Snowpipe loads into staging
CREATE PIPE raw_pipe AS
  COPY INTO staging.orders FROM @my_s3_stage FILE_FORMAT = (TYPE='CSV');

-- Step 2: Stream on staging table
CREATE STREAM orders_stream ON TABLE staging.orders;

-- Step 3: Task to process stream every 5 minutes
CREATE TASK process_orders_task
  WAREHOUSE = 'MEDIUM_WH'
  SCHEDULE = '5 MINUTE'
  WHEN SYSTEM$STREAM_HAS_DATA('orders_stream')
AS
MERGE INTO final.orders_dim t
USING orders_stream s ON t.order_id = s.order_id
WHEN MATCHED THEN UPDATE SET t.status = s.status
WHEN NOT MATCHED THEN INSERT (order_id, status) VALUES (s.order_id, s.status);

-- Step 4: Start the task
ALTER TASK process_orders_task RESUME;
```

■ *SYSTEM$STREAM_HAS_DATA() prevents the task from running when there is nothing to process — saves compute cost.*

N. Top Snowflake Interview Questions — Quick Revision

Use this as a final checklist before your interview. Cover every question confidently.

Q1. What is Snowflake and how is it different from traditional databases?

→ Cloud-native SaaS DWaaS. Separates storage and compute. Runs on AWS/Azure/GCP. No infrastructure management. Supports structured + semi-structured data.

Q2. Explain Snowflake's 3-layer architecture.

→ Storage Layer (micro-partitioned columnar data) → Query Processing Layer (virtual warehouses) → Cloud Services Layer (auth, metadata, optimisation).

Q3. What is a Virtual Warehouse? How is it different from a database?

→ VWH = compute resource. Database = storage container. VWH runs queries but stores no data. You can run multiple VWHs on the same data simultaneously.

Q4. What is the difference between Scale Up and Scale Out?

→ Scale Up: bigger warehouse (more compute per query). Scale Out: more clusters (handles more concurrent users). Scale Out requires Enterprise edition.

Q5. What is Result Caching? When does it NOT work?

→ Snowflake caches results for 24 hours. Cache hit = free, no warehouse needed. Cache miss if: data changed, different user/role, non-deterministic functions (CURRENT_TIMESTAMP, RANDOM).

Q6. What is a Stream and how does it work?

→ CDC object that tracks changes (INSERT/UPDATE/DELETE) on a table. Stores an offset, not data. Has metadata columns: METADATA$ACTION, METADATA$ISUPDATE, METADATA$ROW_ID.

Q7. Difference between Stored Procedure and UDF?

→ UDF: used in SELECT, returns a value, no DML. Stored Procedure: called with CALL, can do DML, used for workflows.

Q8. What is Zero-Copy Clone? Is there any cost?

→ Instantly clones a table/schema/DB without duplicating storage. Clone shares original storage — cost only incurred when changes are made to either object.

Q9. Explain Time Travel and Fail-Safe.

→ Time Travel: user-accessible historical data, up to 90 days (Enterprise). Fail-Safe: 7-day window AFTER Time Travel, managed by Snowflake Support only, last resort.

Q10. What are Micro-Partitions?

→ 50–500 MB columnar storage units. Snowflake stores min/max/NULL metadata per partition. Enables partition pruning — only relevant partitions are scanned.

Q11. What is Snowpipe? How is it different from COPY INTO?

→ Snowpipe: continuous, serverless, event-triggered ingestion — near real-time. COPY INTO: manual/scheduled bulk load — requires a running warehouse.

Q12. What is a Clustering Key and when should you use it?

→ Physically organises micro-partitions by a column for better pruning. Use on very large tables with frequent filters on that column. Not useful for small tables.

Q13. What is Dynamic Data Masking?

→ Masks sensitive column values at query time based on role. Data is stored unmasked — masking is applied dynamically. Used for PII/PHI compliance.

Q14. What is a MERGE statement? When do you use it?

→ Combines INSERT, UPDATE, DELETE in one statement (upsert). Used in ELT pipelines to apply CDC changes from a Stream into a target table.

Q15. What is the QUALIFY clause?

→ Snowflake-specific clause to filter rows based on window function results — like HAVING but for analytical functions. Avoids needing a subquery.

Q16. What are the types of Snowflake tables?

→ Permanent (full TT + FS), Temporary (session-only, no FS), Transient (no FS, cheaper), External (query data outside Snowflake from S3/Azure/GCS).

Q17. Explain the GRANT hierarchy for giving a user table access.

→ GRANT USAGE on DATABASE → GRANT USAGE on SCHEMA → GRANT SELECT on TABLE → GRANT ROLE to USER. All 4 steps required.

Q18. What is a Task Tree / DAG in Snowflake?

→ Tasks can be chained — one root task triggers child tasks on completion. Forms a Directed Acyclic Graph (DAG) for complex workflow orchestration.

This document contains all 12 days of Innova Solutions Snowflake training + interview-critical additions covering Virtual Warehouse sizing, Result Caching, Micro-Partitions, Clustering Keys, Snowflake Editions, GRANT/REVOKE syntax, MERGE statement, Semi-Structured Data (VARIANT/FLATTEN), Dynamic Data Masking, Snowflake-specific SQL functions, Query Profile, UDFs, CTEs, the classic ELT pipeline pattern, and 18 top interview questions with answers. Reference: **docs.snowflake.com**